COMPSCI 589 – Final Project

Benchmarking Classical ML Algorithms on Four Datasets

Vishal Hanuman Akshith Karthik

May 8, 2026

Note on Implementations

All machine learning algorithms were implemented from scratch. No machine learning libraries (e.g., scikit-learn, PyTorch) were used for training or prediction. `numpy` and `matplotlib` were used for numerical computation and plotting. `scikit-learn` was used *only* for dataset loading, not for training or prediction.

- **Datasets 1–2** (Digits, Parkinson's): k-NN, Gaussian Naive Bayes, Random Forest, and Neural Network implementations by **Vishal Hanuman** (HW1–HW4).
- **Datasets 3–4** (Rice, Credit Approval): Experiments by **Akshith Karthik**. Rice uses k-NN and Decision Tree implementations from Akshith Karthik's homework code. Credit Approval uses Akshith Karthik's Decision Tree and Random Forest implementations, with the shared HW4 Neural Network used for the additional neural-network comparison.

1 Introduction

The goal of this project is to compare the performance of classical machine learning algorithms on four benchmark datasets by evaluating accuracy and F1-score under stratified 10-fold cross-validation, with systematic hyperparameter tuning.

2 Datasets

1. **Digits** – 1,797 instances, 64 numerical features, 10 classes (0–9). Loaded using the Digits loader from `scikit-learn`.
2. **Parkinson's Disease** – 195 instances, 22 numerical features, binary (0 = healthy, 1 = Parkinson's).
3. **Rice Grains** – 3,810 instances, 7 numerical features, binary (Cammeo vs. Osmancik).
4. **Credit Approval** – 653 instances, 6 numerical + 9 categorical features, binary (approved vs. not).

3 Experiments and Analyses

3.1 Dataset 1 – Hand-Written Digits

For the Digits dataset, we evaluated k-NN, Random Forest, and a Neural Network. All features are numerical pixel intensities, so no feature encoding was required. k-NN is a natural baseline because digits with similar pixel layouts should be close in the normalized feature space. Random Forests were tested because they can learn nonlinear decision rules over pixel thresholds, while the Neural Network was included because it can combine many pixel-level signals through hidden units. All results below use stratified 10-fold cross-validation and report accuracy and macro F1-score.

k-NN Hyperparameter Sweep

Table 1: Digits: k-NN performance for different values of k.

k	Accuracy	Macro F1
1	0.9872	0.9871
3	0.9872	0.9872
5	**0.9883**	**0.9883**
7	0.9861	0.9861
11	0.9794	0.9793
15	0.9806	0.9805
21	0.9745	0.9743
31	0.9677	0.9674

Random Forest Hyperparameter Sweep

Table 2: Digits: Random Forest performance for different values of *ntree*.

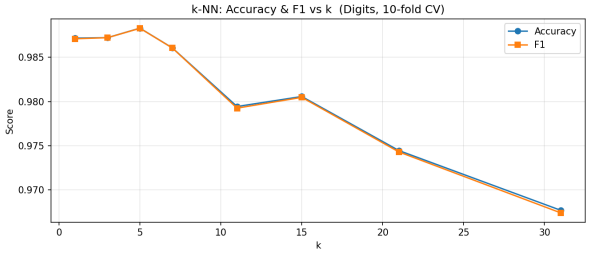
ntree	Accuracy	Macro F1
5	0.8965	0.8957
10	0.9377	0.9373
20	0.9605	0.9604
30	0.9621	0.9620
50	0.9694	0.9695
75	0.9716	0.9718
100	0.9660	0.9660
150	**0.9749**	**0.9748**

Neural Network Hyperparameter Sweep

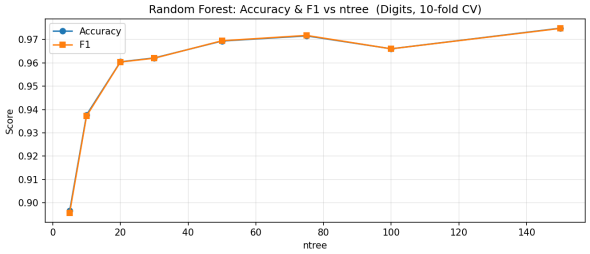
Table 3: Digits: Neural Network performance for selected architectures and regularization values.

Hidden layers	λ	Accuracy	Macro F1
`(32,)`	0.0	0.9800	0.9798
`(64,)`	0.0	**0.9800**	**0.9799**
`(32, 16)`	0.0	0.9756	0.9755
`(64, 32)`	0.0	0.9761	0.9761
`(32,)`	0.1	0.9544	0.9542
`(64, 32)`	0.1	0.9538	0.9536

Learning Curves

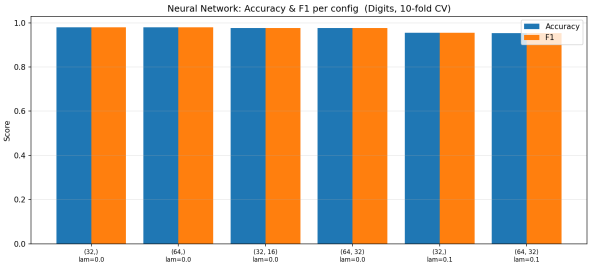


(a) k-NN: Accuracy and F1 vs k

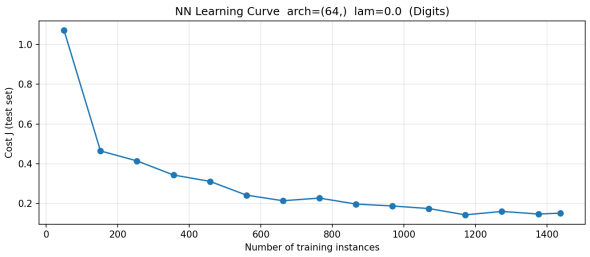


(b) Random Forest: Accuracy and F1 vs *ntree*

Figure 1: Hyperparameter curves for Dataset 1 (Digits).



(a) Neural Network: Accuracy and F1 by configuration



(b) Neural Network: Cost vs training set size

Figure 2: Neural Network graphs for Dataset 1 (Digits).

k-NN achieved the best Digits performance, with $k = 5$ producing 0.9883 accuracy and 0.9883 macro F1. Very small values of k already worked well, which suggests that the normalized pixel space has strong local class structure. Performance declined as k became larger, likely because broad neighborhoods smooth over class boundaries between visually similar digits.

Random Forest performance improved substantially as the number of trees increased, but it remained below k-NN and the Neural Network. This is reasonable for image-like data: an axis-aligned tree split only sees one pixel feature at a time, so the forest needs many trees to approximate

spatial digit patterns. The Neural Network performed strongly, with the best setting at one hidden layer of 64 units and no regularization. Adding regularization with $\lambda = 0.1$ reduced performance, which indicates that these small networks were more likely underfitting than overfitting in this experiment. The learning curve decreased as more training examples were used and then began to flatten, suggesting that the selected network was converging but had less advantage over the simpler k-NN classifier on this dataset.

3.2 Dataset 2 – Parkinson's Disease

For the Parkinson's dataset, we evaluated k-NN, Gaussian Naive Bayes, Random Forest, and a Neural Network. The dataset contains 22 numerical voice-measurement features and a binary target variable, `Diagnosis`. k-NN tests whether nearby voice measurements identify similar diagnoses. Naive Bayes provides a simple probabilistic baseline, Random Forests capture nonlinear feature thresholds, and the Neural Network tests whether a learned nonlinear representation improves performance. The F1-score reported here is the F1-score for the positive Parkinson's class.

k-NN Hyperparameter Sweep

Table 4: Parkinson's: k-NN performance for different values of k.

k	Accuracy	F1
1	**0.9544**	**0.9676**
3	0.9278	0.9530
5	0.9278	0.9528
7	0.9325	0.9565
11	0.8917	0.9319
15	0.8420	0.9034
21	0.8373	0.9028
31	0.8403	0.9047

Gaussian Naive Bayes Hyperparameter Sweep

Table 5: Parkinson's: Gaussian Naive Bayes performance for different variance floors.

ϵ	Accuracy	F1
`1e-9`	0.6915	0.7423
`1e-7`	0.6968	0.7479
`1e-5`	0.6979	0.7504
`0.001`	0.6929	0.7497
`0.01`	0.7129	0.7651
`0.1`	**0.7171**	**0.7695**

Random Forest Hyperparameter Sweep

Table 6: Parkinson's: Random Forest performance for different values of *ntree*.

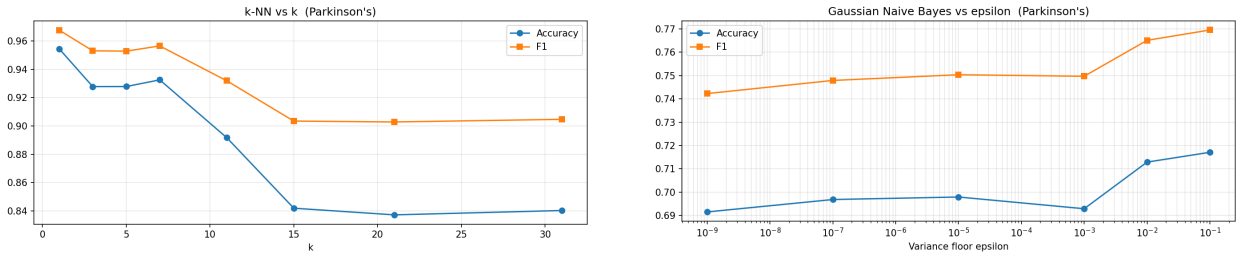
ntree	Accuracy	F1
5	0.8554	0.9055
10	0.8817	0.9245
20	**0.9014**	**0.9376**
30	0.8878	0.9286
50	0.8920	0.9327
75	0.8823	0.9250
100	0.8928	0.9324
150	0.8911	0.9311

Neural Network Hyperparameter Sweep

Table 7: Parkinson's: Neural Network performance for selected architectures and regularization values.

Hidden layers	λ	Accuracy	F1
`(8,)`	0.0	0.8323	0.8920
`(16,)`	0.0	**0.8378**	**0.8968**
`(8, 4)`	0.0	0.7648	0.8652
`(16, 8)`	0.0	0.7659	0.8627
`(8,)`	0.1	0.8303	0.8966
`(16, 8)`	0.1	0.7542	0.8598

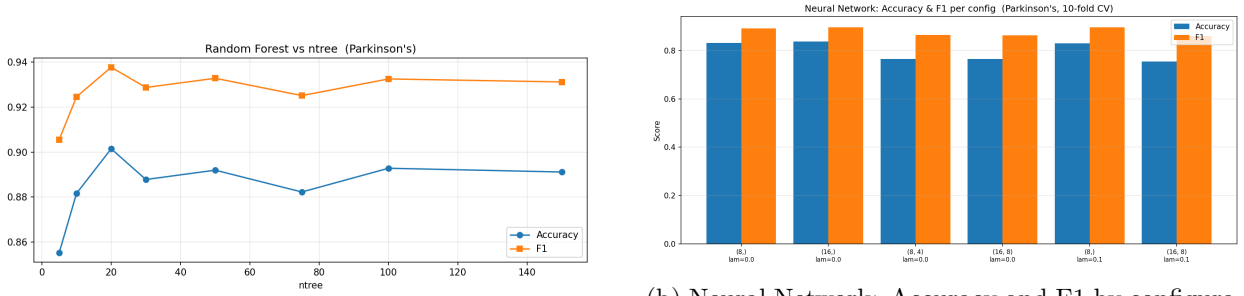
Learning Curves



(a) k-NN: Accuracy and F1 vs k

(b) Naive Bayes: Accuracy and F1 vs ϵ

Figure 3: Hyperparameter curves for k-NN and Naive Bayes on Dataset 2.



(a) Random Forest: Accuracy and F1 vs *ntree*

(b) Neural Network: Accuracy and F1 by configuration

Figure 4: Random Forest and Neural Network configuration curves for Dataset 2.

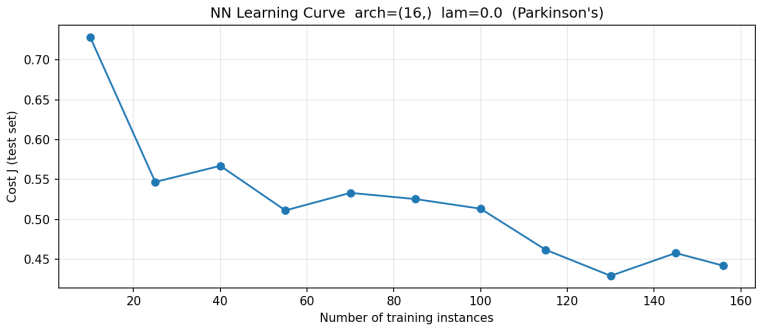


Figure 5: Neural Network learning curve for Dataset 2.

k-NN achieved the best Parkinson's performance, with $k = 1$ producing 0.9544 accuracy and 0.9676 F1. Larger values of k reduced performance, which suggests that the most useful signal is very local in the normalized voice-feature space. Because the dataset is small and imbalanced toward the positive class, larger neighborhoods likely mix less similar patients and smooth away useful diagnostic differences.

Gaussian Naive Bayes was the weakest model. The voice measurements are likely correlated with each other, while Naive Bayes assumes conditional independence given the class. The epsilon sweep shows that increasing the variance floor helped slightly, but it did not close the gap to the other methods. Random Forest was the second-best method: it handled nonlinear feature thresholds better than Naive Bayes, but performance peaked at 20 trees and then varied slightly, which is consistent with the higher variance expected from bootstrap training on only 195 instances. The Neural Network trailed k-NN and Random Forest; the deeper architectures were worse, suggesting that the dataset is too small for the added capacity. Its learning curve generally decreased as more training instances were used, but the curve was not perfectly smooth because the train/test split is small and the training procedure is stochastic.

3.3 Dataset 3 – Rice Grains

For the Rice Grains dataset, we evaluated k-NN and a Decision Tree. The dataset has 3,810 examples with seven numerical shape descriptors and a binary label (`Cammeo` or `Osmancik`). Since all features are numerical, k-NN can use normalized distances directly. The Decision Tree provides a comparison based on learned feature-threshold rules. We used stratified 10-fold cross-validation and reported accuracy and F1-score.

k-NN Hyperparameter Sweep

Table 8: Rice: k-NN performance for different values of k.

k	Accuracy	F1
1	0.8853	0.8994
3	0.9118	0.9229
5	0.9215	0.9315
7	0.9236	0.9333
9	0.9239	0.9336
11	**0.9249**	**0.9345**

Decision Tree Hyperparameter Sweep

Table 9: Rice: Decision Tree performance for different maximum depths.

Max depth	Accuracy	F1
2	**0.9247**	**0.9337**
4	0.9231	0.9329
6	0.9134	0.9246
8	0.9108	0.9221
10	0.9052	0.9175

Learning Curves

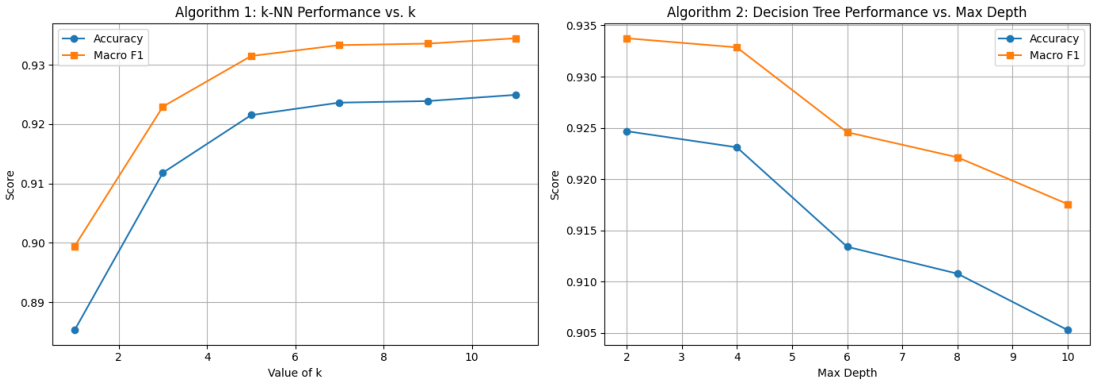


Figure 6: Rice: k-NN and Decision Tree hyperparameter sweeps.

Both methods performed strongly on Rice. k-NN with $k = 11$ achieved the best overall result, with 0.9249 accuracy and 0.9345 F1. The steady improvement from $k = 1$ through $k = 11$ suggests that local neighborhoods are helpful, but the dataset benefits from some smoothing instead of relying on a single nearest neighbor. The Decision Tree was almost as strong at depth 2, with 0.9247 accuracy and 0.9337 F1. Deeper trees performed worse, which suggests that a small number of

shape-feature thresholds captures most of the useful class separation and that additional depth begins to fit fold-specific noise.

3.4 Dataset 4 – Credit Approval

For the Credit Approval dataset, we evaluated Decision Tree, Random Forest, and Neural Network models. This dataset is different from the first three because it mixes numerical and categorical attributes. The categorical variables were converted into one-hot features before model training, while numerical variables were kept as continuous values. We again used stratified 10-fold cross-validation and compared accuracy and F1-score.

Decision Tree Hyperparameter Sweep

Table 10: Credit Approval: Decision Tree performance for different maximum depths.

Max depth	Accuracy	F1
2	**0.8636**	**0.8627**
4	0.8255	0.8152
6	0.8317	0.8110
8	0.8318	0.8161
10	0.8181	0.7989
15	0.8148	0.7962

Random Forest Hyperparameter Sweep

Table 11: Credit Approval: Random Forest performance for different values of *ntree*.

ntree	Accuracy	F1
1	0.8086	0.7852
5	0.8547	0.8393
10	0.8698	0.8586
20	0.8650	0.8551
30	**0.8728**	**0.8637**

Neural Network Hyperparameter Sweep

Table 12: Credit Approval: Neural Network performance for selected architectures and regularization values.

Hidden layers	λ	Accuracy	F1
`(8,)`	0.0	0.8608	0.8456
`(16,)`	0.0	0.8484	0.8313
`(8, 4)`	0.0	0.8470	0.8271
`(16, 8)`	0.0	0.8547	0.8367
`(8,)`	0.1	**0.8623**	**0.8552**
`(16, 8)`	0.1	0.5467	0.0000

Learning Curves

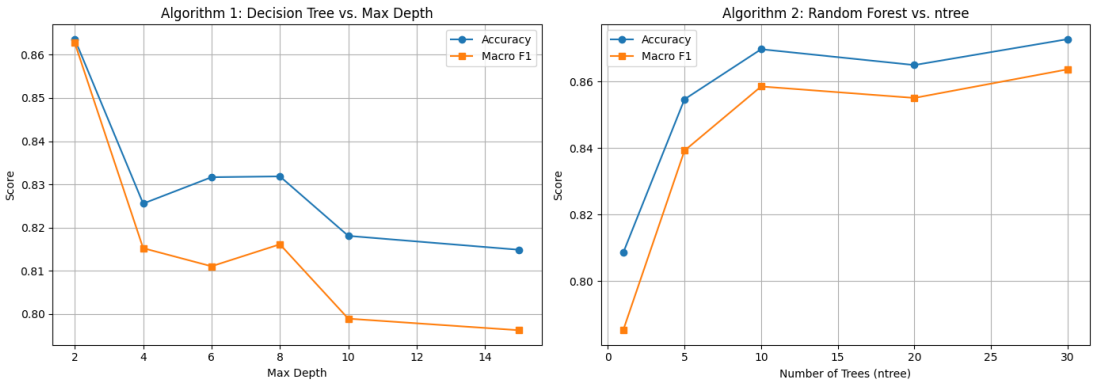
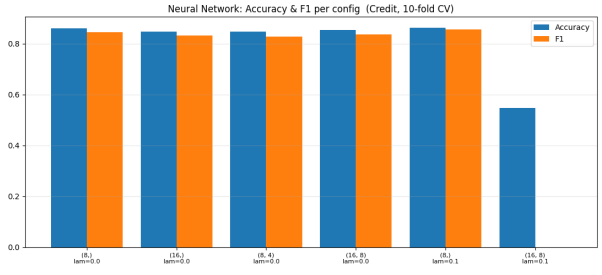
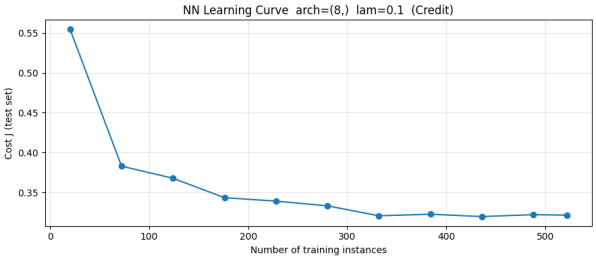


Figure 7: Credit Approval: Decision Tree and Random Forest hyperparameter sweeps.



(a) Neural Network: Accuracy and F1 by configuration



(b) Neural Network: Cost vs training set size

Figure 8: Credit Approval: Neural Network configuration and learning curves.

Random Forest gave the strongest Credit Approval result, with *ntree*=30 producing 0.8728 accuracy and 0.8637 F1. The single Decision Tree was close when kept shallow: depth 2 reached 0.8636 accuracy and 0.8627 F1, but deeper trees declined. That pattern suggests that the dataset has a few strong approval rules, while deeper splits begin to memorize smaller subgroups. The Random

Forest improved over a single tree by averaging many bootstrapped trees, which helped stabilize predictions on this mixed-feature dataset. The Neural Network was competitive but slightly behind the forest. The best network used one hidden layer of 8 units with $\lambda = 0.1$, reaching 0.8623 accuracy and 0.8552 F1. The larger regularized architecture collapsed to an F1 of 0.0000, which likely means it predicted mostly one class under that setting. For this dataset, the tree-based models were a better fit than the deeper neural-network configurations.

4 Summary of Results

Table 13: Best performance (Accuracy / F1-Score) per algorithm per dataset. Gray cells indicate the highest value for each metric on each dataset.

	Digits		**Parkinson's**		**Rice**		**Credit**	
Algorithm	Acc	F1	Acc	F1	Acc	F1	Acc	F1
k-NN	**0.9883**	**0.9883**	**0.9544**	**0.9676**	**0.9249**	**0.9345**	–	–
Decision Tree	–	–	–	–	0.9247	0.9337	0.8636	0.8627
Naive Bayes	–	–	0.7171	0.7695	–	–	–	–
Random Forest	0.9749	0.9748	0.9014	0.9376	–	–	**0.8728**	**0.8637**
Neural Network	0.9800	0.9799	0.8378	0.8968	–	–	0.8623	0.8552

5 Discussion

Across the four main datasets, no single model dominated every setting, but the patterns were consistent with the structure of the data. k-NN was strongest on Digits, Parkinson's, and Rice. Those datasets all have numerical feature spaces where distance is meaningful after normalization. Digits and Rice especially show that local neighborhoods carry strong class information.

Tree-based methods were strongest on Credit Approval. That dataset mixes categorical and numerical features, and the approval label appears to depend on a small set of rule-like conditions. A shallow Decision Tree already performed well, and the Random Forest improved slightly by averaging several bootstrapped trees. In contrast, deeper single trees became worse on both Rice and Credit, which points to overfitting rather than underfitting.

The Neural Network was competitive when there were enough examples and a useful continuous feature representation, especially on Digits. It was weaker on Parkinson's and Credit. For Parkinson's, the dataset is very small, so deeper networks add capacity without much data to support it. For Credit, the one-hot feature representation is sparse and the tree-based models fit the rule-like structure more naturally. Naive Bayes was the weakest model where it was tested, which is reasonable because the Parkinson's voice measurements are likely correlated and violate the model's conditional-independence assumption.

6 Extra Credit

[EC#2] Image Classification on Olivetti Faces

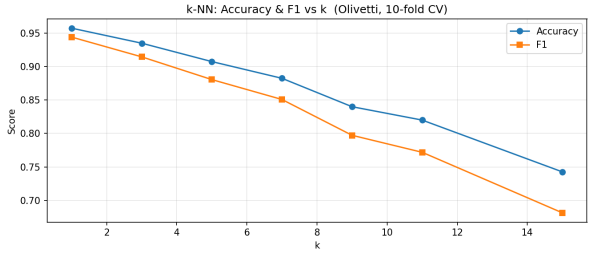
To satisfy the EC#2 criterion of selecting a new *challenging* dataset, we evaluated our k-NN (HW1) and Neural Network (HW4) implementations on the **Olivetti Faces** dataset — 400 grayscale 64x64 images of 40 distinct subjects (10 images per subject), loaded with a `scikit-learn` dataset fetcher.

This is an image classification task with 40 classes, far more than any of the four main project datasets. The full notebook is at `notebooks/05_olivetti_ec2.ipynb`.

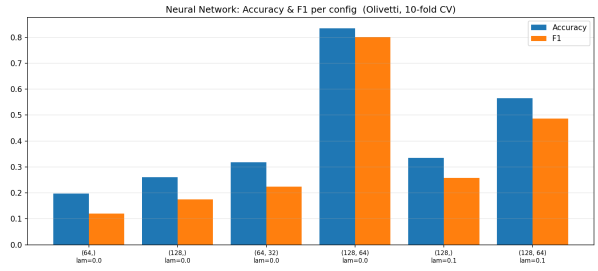
Random Forest was omitted because the from-scratch implementation is too slow with 4096 pixel features. k-NN and Neural Network are also strong face-recognition baselines.

Table 14: Olivetti Faces (EC#2): best result per algorithm, stratified 10-fold CV.

Algorithm	Best Hyperparameter	Accuracy	Macro F1
k-NN	$k = 1$	**0.9575**	**0.9442**
Neural Network	`(128, 64)`, $\lambda = 0.0$	0.8350	0.8011



(a) k-NN: Accuracy and F1 vs k



(b) Neural Network: Accuracy and F1 by configuration

Figure 9: Olivetti Faces (EC#2): hyperparameter sweeps.

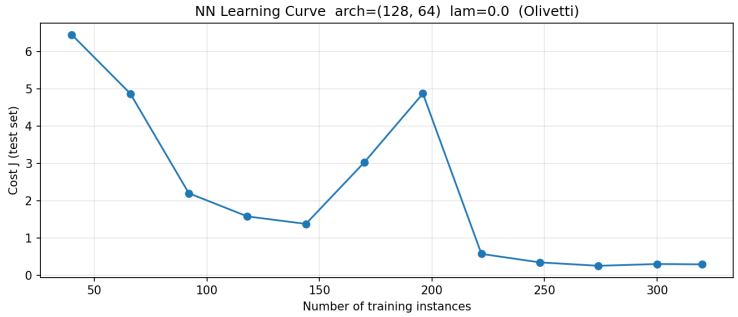


Figure 10: Olivetti Faces (EC#2): NN learning curve.

The Olivetti results support the extra-credit motivation: this is a small but difficult 40-class image-classification task. k-NN achieved the best result, with $k = 1$ reaching 0.9575 accuracy and 0.9442 macro F1. Performance decreased as k increased, which suggests that each subject's nearest images are highly informative but larger neighborhoods begin mixing other identities. The Neural Network also learned a useful classifier, with its best architecture reaching 0.8350 accuracy and 0.8011 macro F1, but it was weaker than k-NN. This is expected because the dataset has only 400 images and 40 classes, so the network has limited data for learning a stable face representation from 4096 pixel-level inputs.